Introducción a Apache Hadoop

Francisco Javier Sueza Rodríguez

30 de octubre de 2026

Centro: IES Zaidin-Vergeles
Curso: Inteligencia Artificial y Big Data
Módulo: Big Data Aplicado

Índice

1 Creación de la Máquina Virtual **4**
1.1 Solución 4

2 Configuración del Nombre del Equipo y Preparación del Sistemas **6**
2.1 Solución 6

3 Creación del Usuario Hadoop **7**
3.1 Solución 7

4 Configuración de SSH **8**
4.1 Solución 8

5 Instalación de Apache Hadoop **9**
5.1 Solución 10

6 Configuración Pseudo-distribuida **11**
6.1 Solución 11

7 Formateo y Puesta en marcha de HDFS y YARN **12**
7.1 Solución 12

8 Interfaces web de Hadoop **13**
8.1 Solución 13

9 Sistemas de almacenamiento Big Data **15**
9.1 Solución 15

10 Creación de la Estructura HDFS **15**
10.1 Solución 16

11 Preparación del conjunto de datos **16**
11.1 Solución 16

12 Ingestión de los Datos **17**
12.1 Solución 17

13 Análisis de formatos de Almacenamiento **18**
13.1 Solución 18

14 Comprobación de almacenamiento físico en HDFS **20**
14.1 Solución 20

15 Procesamiento de los datos **21**
15.1 Solución 21

Referencias **24**

Índice de figuras

1.1 Configuración General de la Imagen 4
1.2 Configuración de CPU/RAM de la Imagen 5
1.3 Configuración del Alamacenamiento y de Red de la Imagen 5
1.4 Primer inicio de Ubuntu Server 6
2.1 Ejecución de los comandos hostname y hostnamectl 6
2.2 Versión de Java instalada 7
3.1 Prompt con el usuario y el hostname 7
4.1 Generación de las claves de ssh 8
4.2 Contenido del fichero autorized_keys 8
4.3 Conexión mediante ssh a localhost 9
4.4 Conexión empleando el host nodo1-fjsr-26-27 9
5.1 Directorio de instalación de Hadoop 10
5.2 Valores de las variables de entorno 10
5.3 Versión de Hadoop instalada 10
6.1 Configuración core-site.xml 11
6.2 Configuración hdfs-site.xml 11
6.3 Configuración mapred-site.xml 12
6.4 Configuración yarn-site.xml 12
7.1 Formateo del namenode 13
7.2 Procesos mostrados con jps 13
8.1 Interfaz web de NameNode 14
8.2 Interfaz web de YARN 14
10.1 Estructura de archivos en HDFS 16
11.1 Conteo de líneas del archivo ventas.csv 16
11.2 Archivo ventas.csv 17
12.1 Archivos locales de datos 17
12.2 Uso de cat y ls en HDFS 18
13.1 Contenido del fichero en formato CSV 19
13.2 Contenido del fichero en formato JSON 19
13.3 Tamaño de los archivos CSV y JSON en local y dentro de HDFS 20
14.1 Información del fichero ventas.csv en la interfaz web de NameNode 20
14.2 Detalle el fichero ventas.csv en la interfaz web de NameNode 21
15.1 Script de bash para procesar los datos 22
15.2 Script de bash para procesar los datos 22

1. Creación de la Máquina Virtual

Crea en Oracle VirtualBox una máquina virtual con Ubuntu Server 26.04 LTS, 2 procesadores virtuales, 8 GB de RAM, 100 GB de almacenamiento y una configuración de red que permita acceso a Internet y a las interfaces web de Hadoop desde el equipo anfitrión.

1.1. Solución

En este primer ejercicio se va a proceder a instalar **Ubunut Server 26.04.1** en una máquina virtual de **VirtualBox**, la cual se ha instalado en el sistema operativo del host, que es una **Kubuntu 26.04.1**, y se ha creado una imagen usando es especificaciones indicadas.

En primer lugar se ha creado la imagen con el nombre **practica1-bda**, esto no estaba especificado en la practica por lo que se ha elegido este para dejar claro su propósito. La imagen se ha creado casi de forma automática, solo especificando algunos datos durante el proceso.

Se ha creado el **usuario hadoop**, tal y como se especificaba en el enunciado, pero la ir a crear le **hostname** con las indicaciones dadas ha surgido un problema, y es que Linux **no admite la barra baja** (_) como carácter para nombrar el hostname, por lo que sea ha sustituido por la barra media. Eso se debe a que según el estándar [1], no se puede emplear este carácter para el hostname.

En la siguiente captura se puede ver la configuración general generada después de crear la imagen:

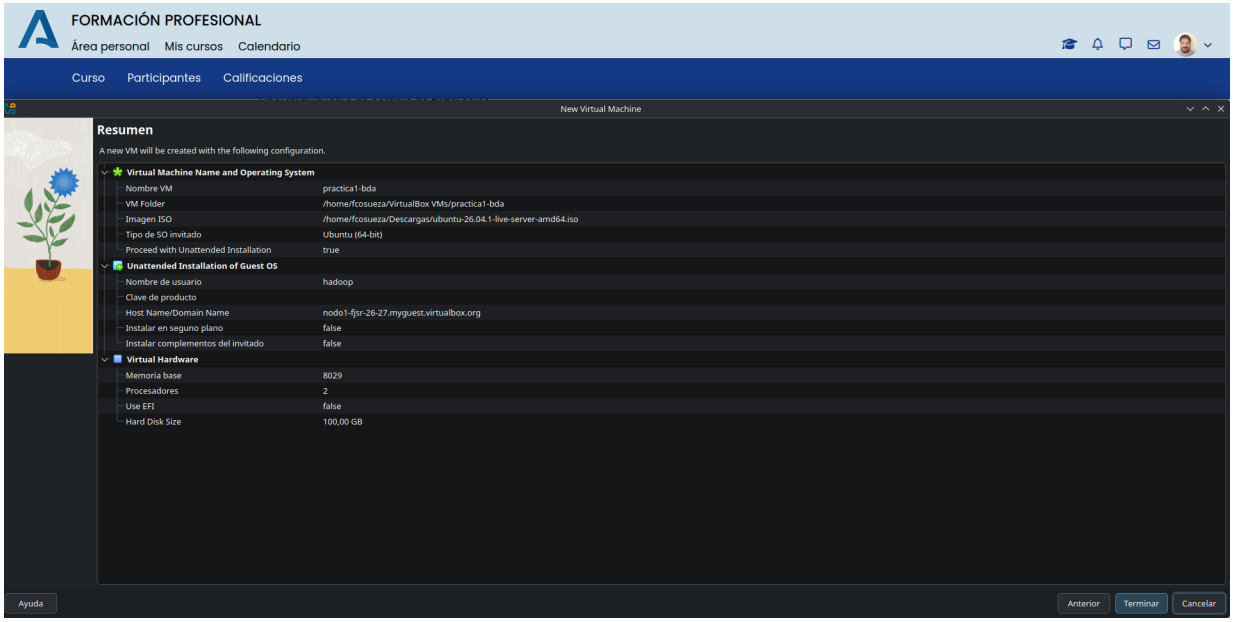


Figura 1.1: Configuración General de la Imagen

Después de realizar la configuración general, se ha elegido el **número de CPUs** que queremos que tenga la imagen, que este caos han sido 2, y la **cantidad de RAM** que va a tener la máquina virtual, que va a ser un total de **8 GB**, tal y como se especifica en el enunciado.

En la siguiente captura, tomada durante el proceso de creación, se puede ver esta información:

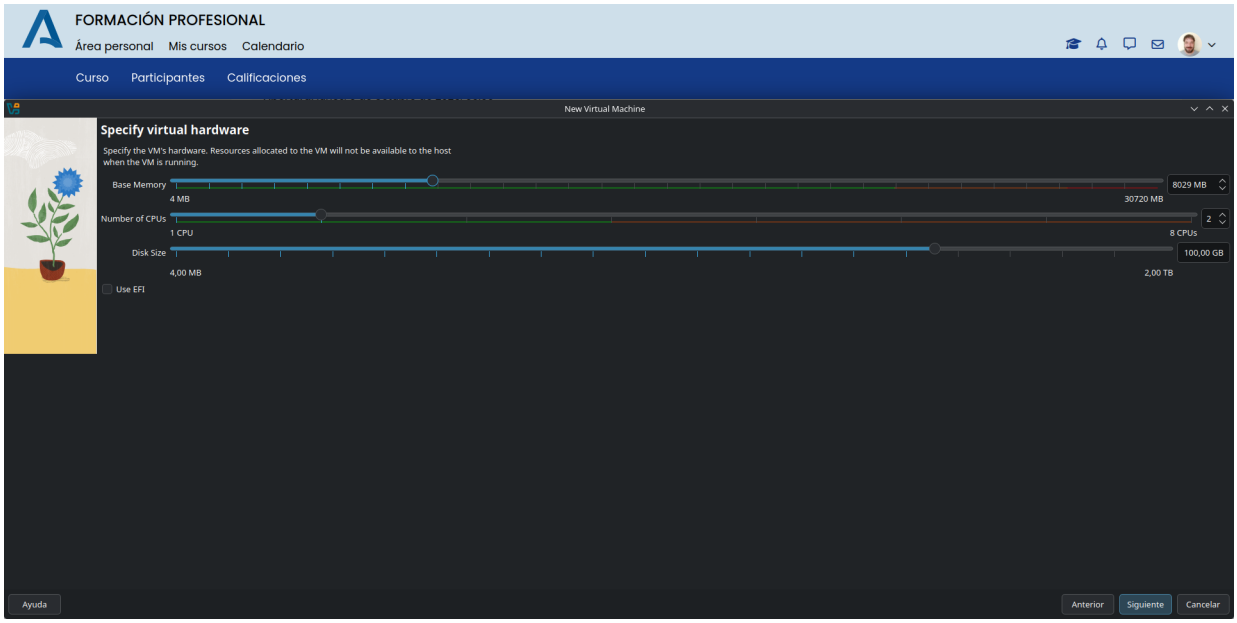


Figura 1.2: Configuración de CPU/RAM de la Imagen

También se ha seleccionado **100 GB de almacenamiento**, como podemos ver en la primera captura. El resto de la configuración se ha creado de forma automática, durante el proceso de creación de la imagen, como por ejemplo, el adaptador de red que se usa. En la siguiente captura, podemos ver la configuración creada tanto del Almacenamiento como de red:

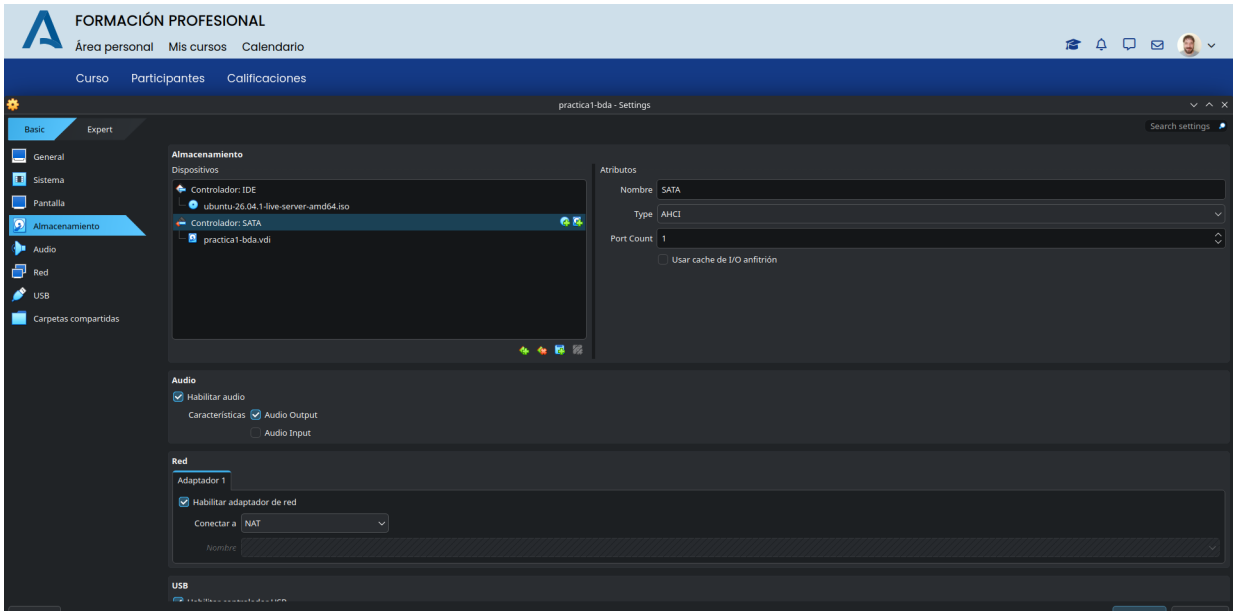


Figura 1.3: Configuración del Alamacenamiento y de Red de la Imagen

Como podemos ver, ha configuración la conexión de red automáticamente con **NAT**, así como el controlador SATA para el almacenamiento.

Una vez realizada la configuración, se ha **instalado Ubuntu Server** en la máquina virtual. Una vez realizada la instalación, se ha **iniciado Ubuntu Server** por primera, como podemos ver a continuación:

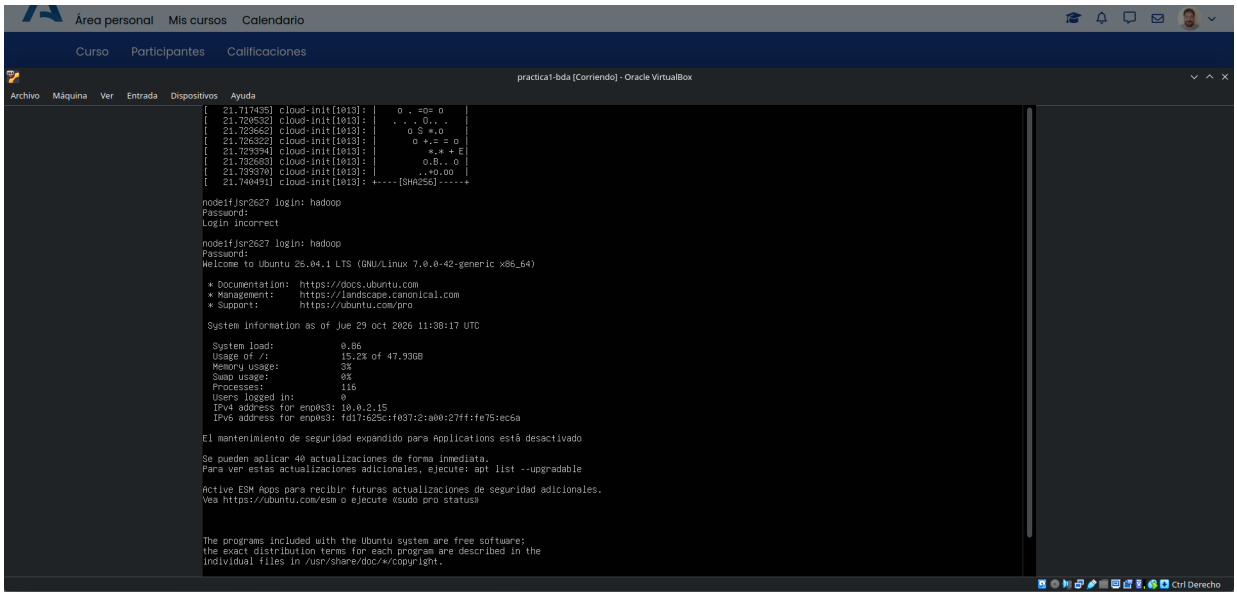


Figura 1.4: Primer inicio de Ubuntu Server

2. Configuración del Nombre del Equipo y Preparación del Sistemas

Configura el nombre del nodo con el formato **nodo1_iniciales_26_27**, compruébalo con **hostname** y **hostnamectl**, actualiza el sistema, verifica la conectividad e instala Java. Explica por qué Hadoop necesita Java y la función de **JAVA_HOME**.

2.1. Solución

Como se comentaba en el punto anterior, **ni Linux**, **ni VirtualBox permiten** los nombres de host con **guiones bajos**, así que se ha optado por reemplazar los guiones bajos por guiones medios, para evitar conflictos. En la siguiente captura, podemos ver la ejecución de los comandos **hostname** y **hostnamectl**.

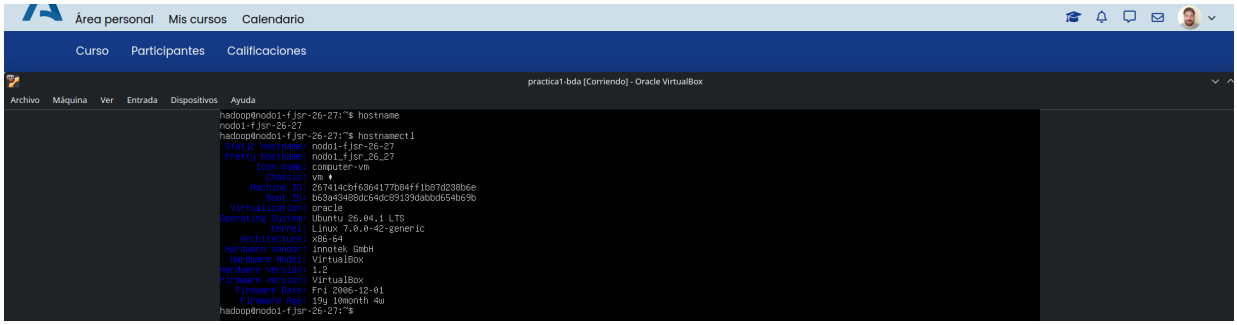


Figura 2.1: Ejecución de los comandos hostname y hostnamectl

A continuación se ha actualizado el sistemas, empleando el comando **sudo apt-get update** y se ha procedido a instalar Java, mediante los paquetes **default-jre** y **default-jdk**, que se han instalado sin problemas. En la siguiente captura, podemos ver la versión instalada de Java:

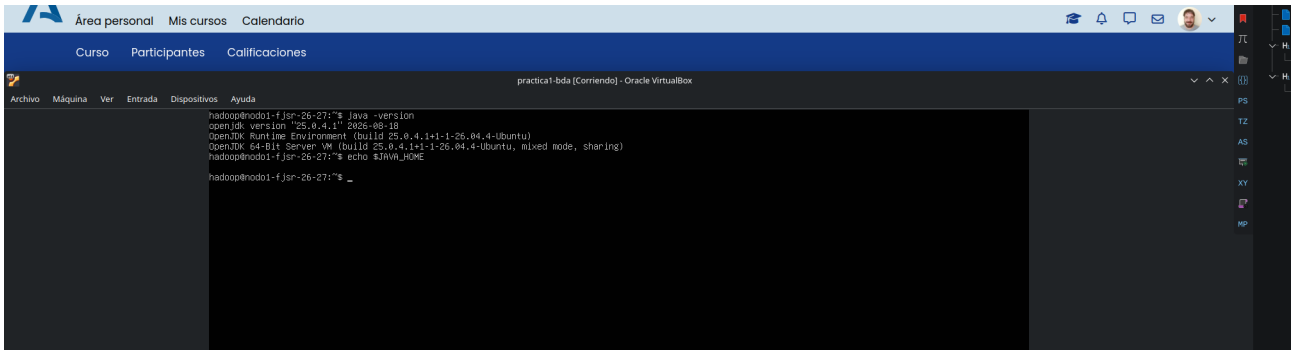


Figura 2.2: Versión de Java instalada

El motivo por que Hadoop es porque **esta desarrollado en Java**, especialmente las librerías principales del proyecto, aunque hay otros lenguajes que se incluyen en otras aplicaciones del ecosistemade Hadoop [2]. Respecto a **JAVA_HOME**, es una variable de entorno importante, ya que apunta al **entorno de ejecución de JAVA**, o **JRE** (Java Runtime Environment).

3. Creación del Usuario Hadoop

Crea el usuario **hadoop**, su directorio personal, configura los permisos necesarios y accede con dicho usuario. El prompt debe tener un formato similar a **hadoop@nodo1_abs_26_27: $**.

3.1. Solución

El usuario hadoop **se ha creado** durante el proceso de **instalación**, por lo que no ha sido necesario crearlo ahora, aunque hubiera sido sencillo usando el comando **adduser**, pero no ha sido necesario. En la siguiente captura se puede ver el prompt correcto.

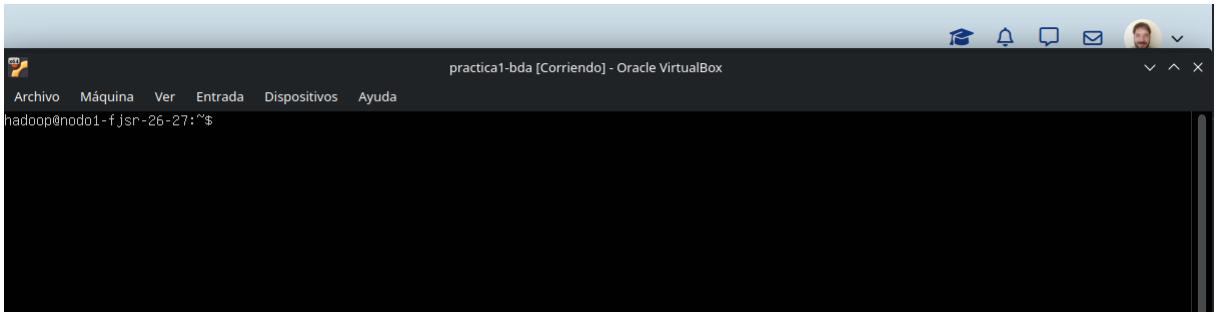


Figura 3.1: Prompt con el usuario y el hostname

4. Configuración de SSH

Instala y configura SSH para que el usuario **hadoop** pueda conectarse sin contraseña mediante clave pública. Prueba **ssh localhost** y **ssh nodo1_iniciales_26_27**. Explica por qué Hadoop utiliza SSH.

4.1. Solución

En este ejercicio vamos a configurar SSH. **Hadoop** necesita ssh ya que este permite la conexión, de forma segura, a todos los nodos de la red distribuida. De esta forma el framework puede realizar operaciones en dichos nodos de forma segura, y aunque nosotros solo vamos a tener un nodo real, vamos a simular una red distribuida, así que necesitamos instalarlo.

No se ha instalado ssh ya que esto se ha realizado durante la instalación. En primer lugar se **han generado las claves** usando el comando **ssh-keygen**, para posteriormente introducir la clave en el fichero **autorized_keys** y cambiar los permisos para que todo funcione correctamente. A continuación se muestran las capturas con la configuración inicial.

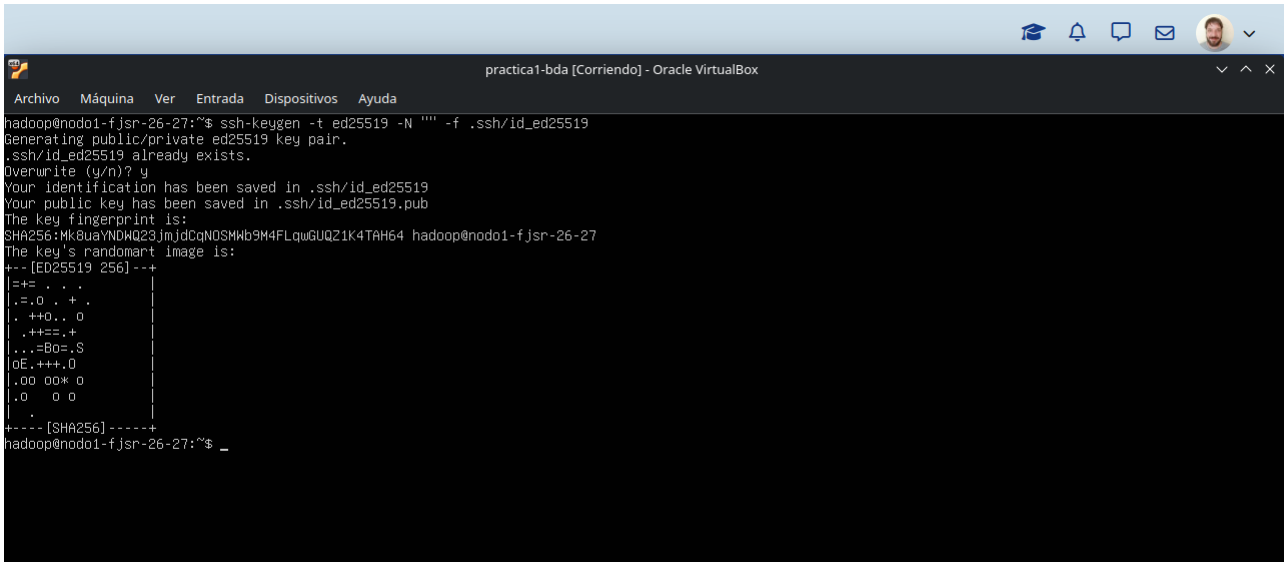


Figura 4.1: Generación de las claves de ssh

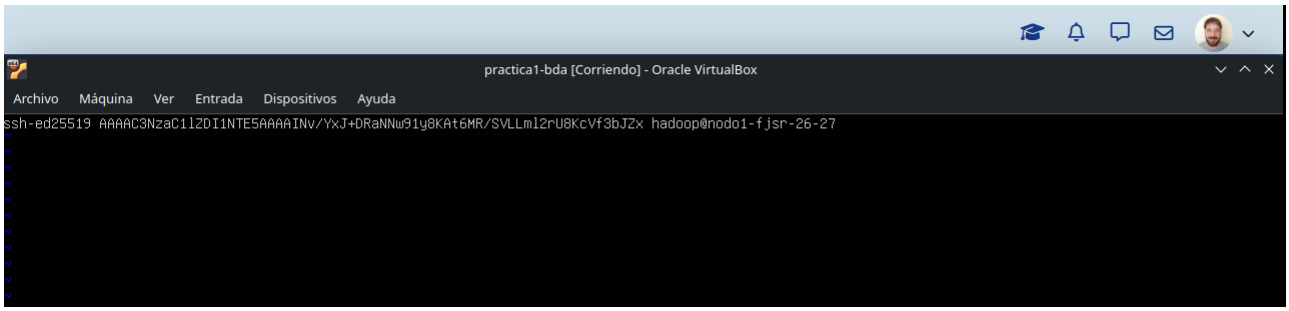


Figura 4.2: Contenido del fichero autorized_keys

Una vez configurado todo, se ha comprobado que funciona correctamente realizando la conexión al sistema empleando los 2 hosts especificados en el enunciado.

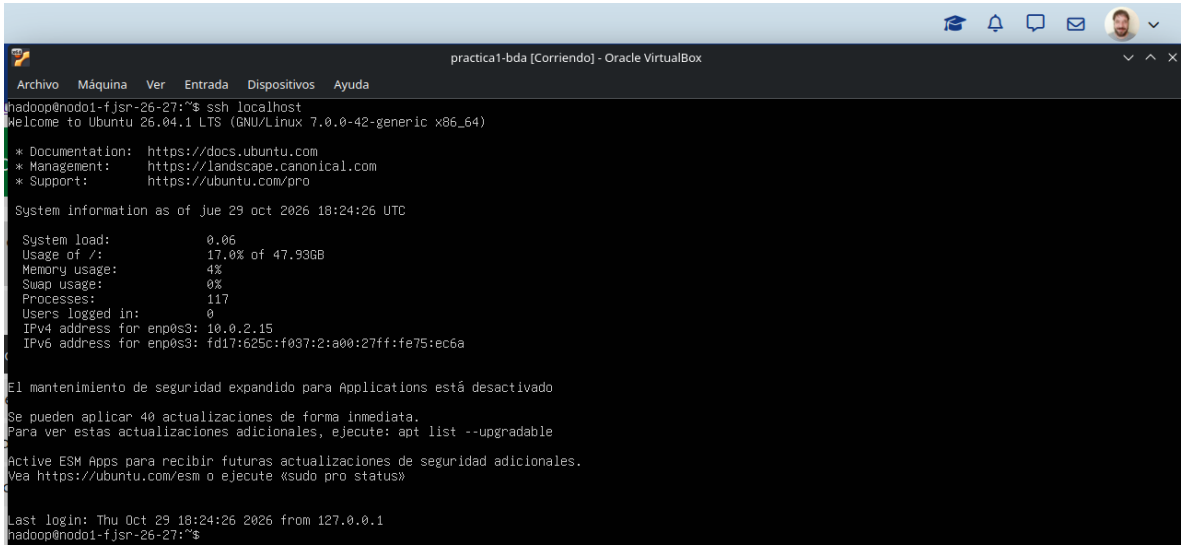


Figura 4.3: Conexión mediante ssh a localhost

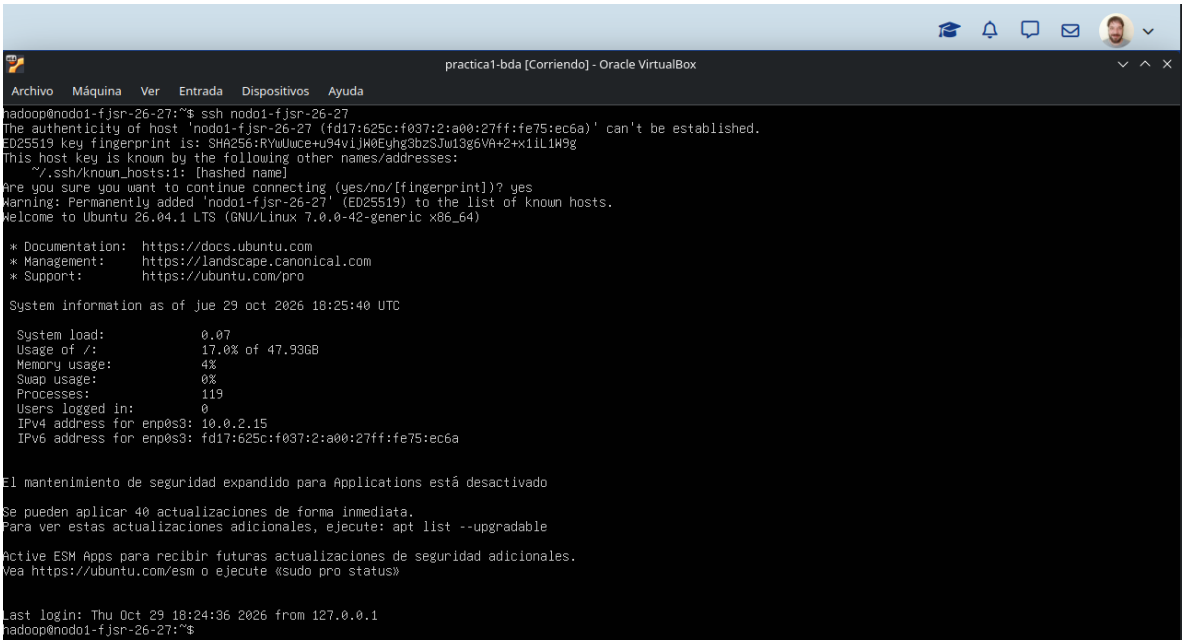


Figura 4.4: Conexión empleando el host nodo1-fjsr-26-27

5. Instalación de Apache Hadoop

Descarga e instala Apache Hadoop en **/opt/hadoop**. Configura **JAVA_HOME**, **HADOOP_HOME**, **HADOOP_CONF_DIR** y **PATH**. Comprueba la instalación con **hadoop version**.

5.1. Solución

En este ejercicio vamos a **instalar Hadoop** y a configurar las variables de entorno para su correcto funcionamiento. Para realizar la instalación, se ha elegido la **distribución binaria** que podemos encontrar en la página de descargar de Hadoop, mediante **wget**, ya que no tenemos interfaz gráfica, y se ha extraído en **/opt**. En la siguiente captura podemos ver el directorio de instalación de Hadoop.

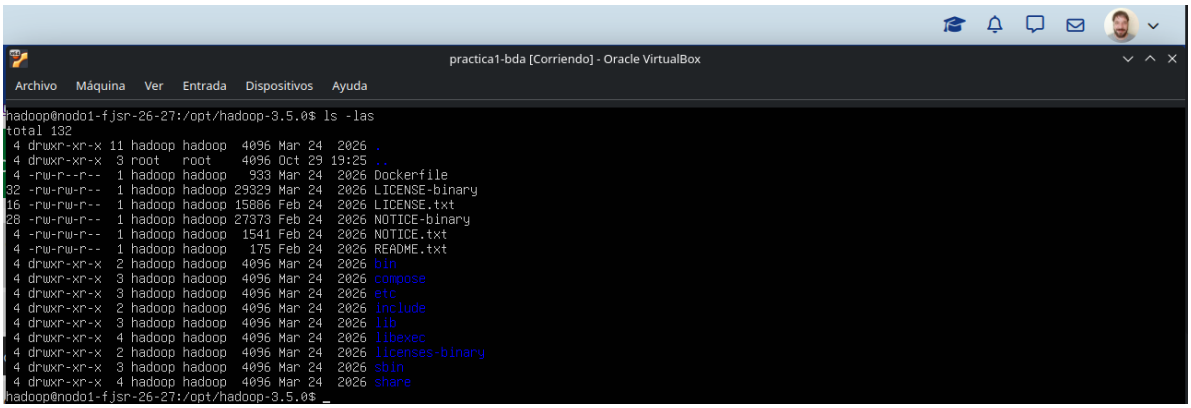


Figura 5.1: Directorio de instalación de Hadoop

A continuación se han establecido todas las variables de entorno para que Hadoop pueda funcionar correctamente, lo que se ha hecho incluyéndolas en el fichero **.bashrc** del directorio home del usuario. Por último, se ha comprobado que la versión de Hadoop es la correcta, como podemos ver en las siguientes capturas.



Figura 5.2: Valores de las variables de entorno



Figura 5.3: Versión de Hadoop instalada

6. Configuración Pseudo-distribuida

Configura **core-site.xml**, **hdfs-site.xml**, **mapred-site.xml** y b para trabajar en modo pseudo-distribuido. Explica la función de cada fichero y justifica **dfs.replication = 1**.

6.1. Solución

En este ejercicio vamos a **configurar hadoop** para porder trabajar en modo pseudo-distribuido. Para ello vamos a modificar una serie de archivos que nos permitirán trabajar de este modo. Los ficheros y sus configuraciones son los siguientes:

- **core-site.xml**: contiene la configuración general de Hadoop y los parámetros comunes a todos sus componentes. En la siguiente captura podemos ver sus parámetros:

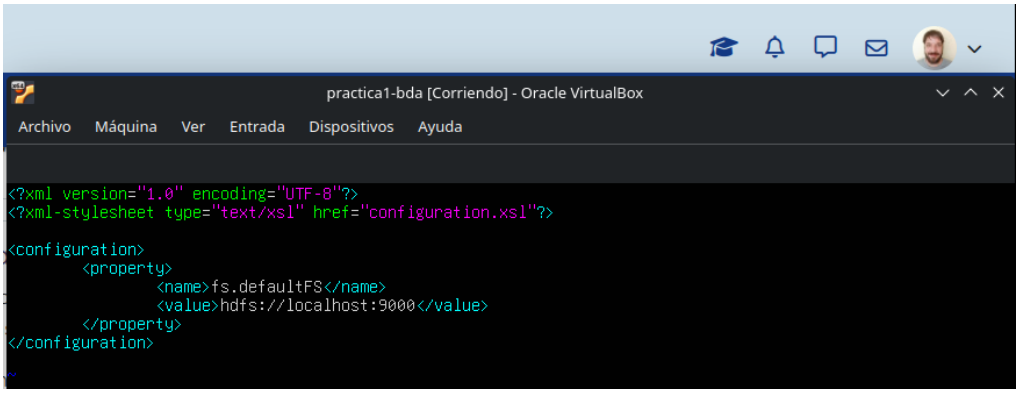


Figura 6.1: Configuración core-site.xml

- **hdfs-site.xml**: contiene la configuración del sistema de ficheros HDFS.

  En este archivo es donde hemos configurado la opción **dfs.replication** con valor 1. Esto se ha realizado así ya que esta configuración se utiliza para indicar a Hadoop **cuantas replicas** debe tener de cada bloque de datos. Por norma genera, cuando tenemos varios nodos suele haber varias replicas, por si falla alguna, pero como aquí solo tenemos un nodo solo podremos tener 1 replica de cada bloque de datos.

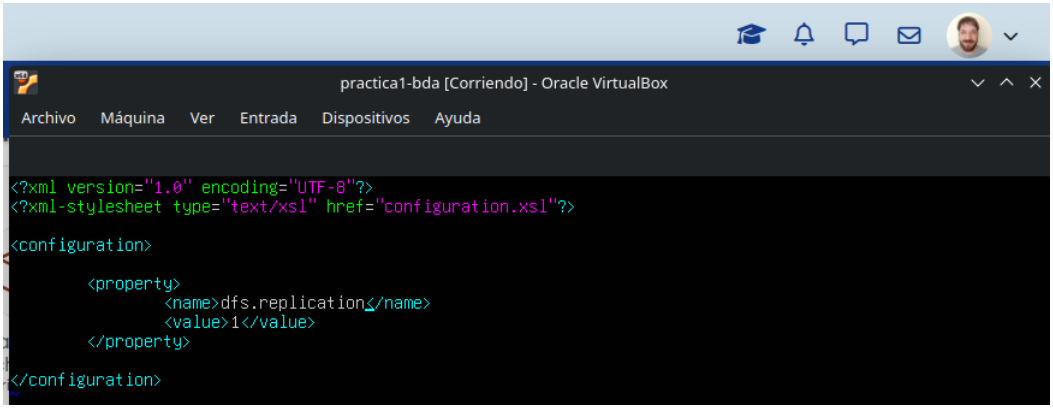


Figura 6.2: Configuración hdfs-site.xml

- **mapred-site.xml**: este archivo permite configurar el MapReduce, que permite dividir los trabajos en tareas y ejecutarlas de forma distribuida. Los valores que hemos configurado se pueden ver en la siguiente captura:

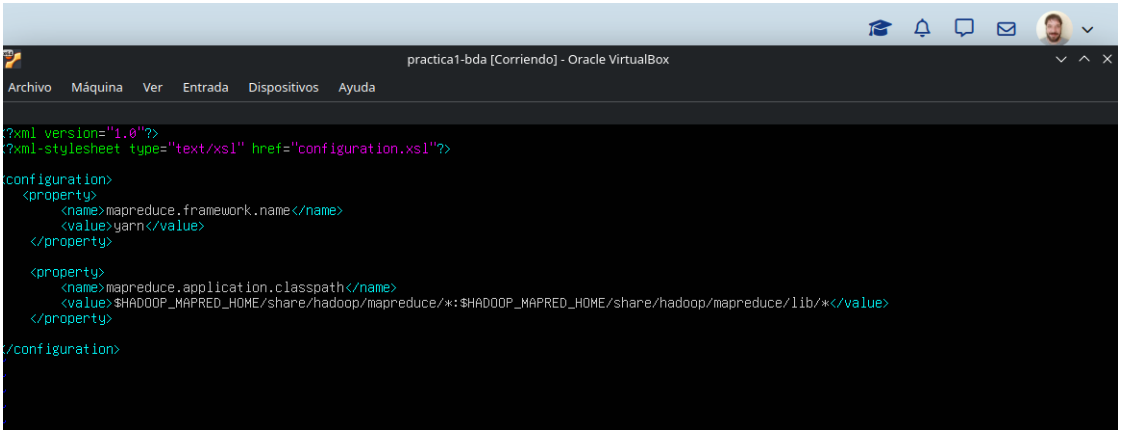


Figura 6.3: Configuración mapred-site.xml

- **yarn-site.xml**: contiene la configuración de YARN, el sistema encargado de gestionar los recursos del la red de nodos y coordinar la ejecución de las aplicaciones.

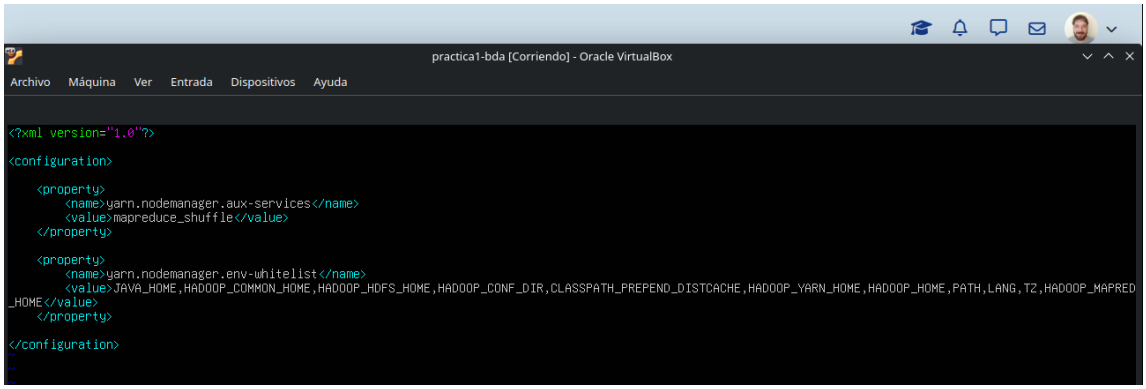


Figura 6.4: Configuración yarn-site.xml

7. Formateo y Puesta en marcha de HDFS y YARN

Formatea el NameNode, inicia HDFS y YARN y comprueba los procesos mediante jps. Deben aparecer, según la configuración, **NameNode**, **DataNode**, **SecondaryNameNode**, **ResourceManager** y **NodeManager**.

7.1. Solución

En es ejercicio vamos a formatear el namenode, para poder iniciar HDFS y así mismo vamos a iniciar también YARN, comprobado que no esta funcionando correctamente, como podemos ver en las siguientes capturas.

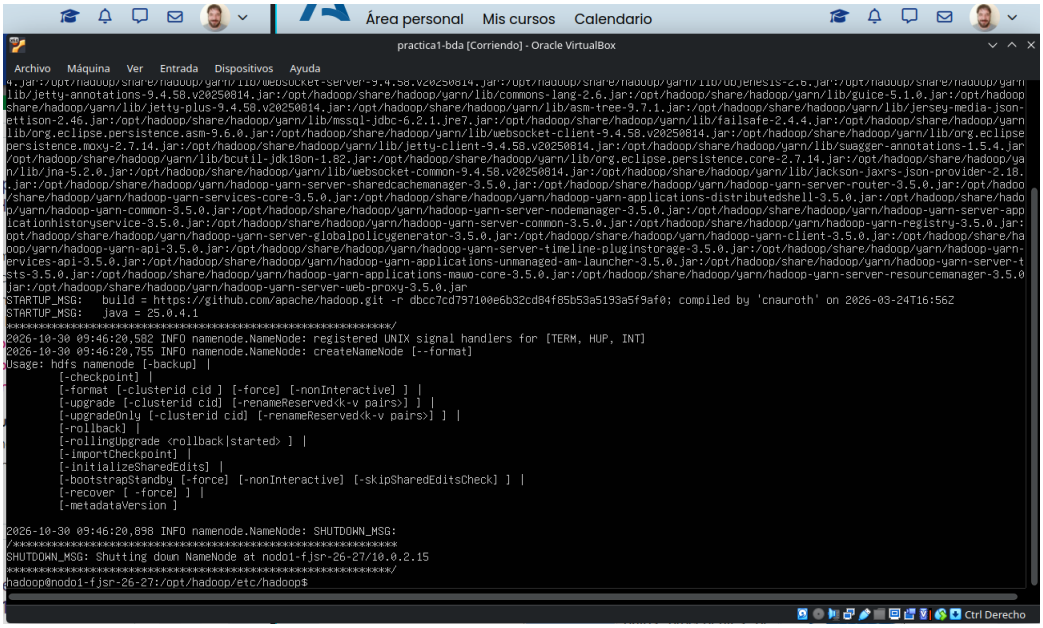


Figura 7.1: Formateo del namenode

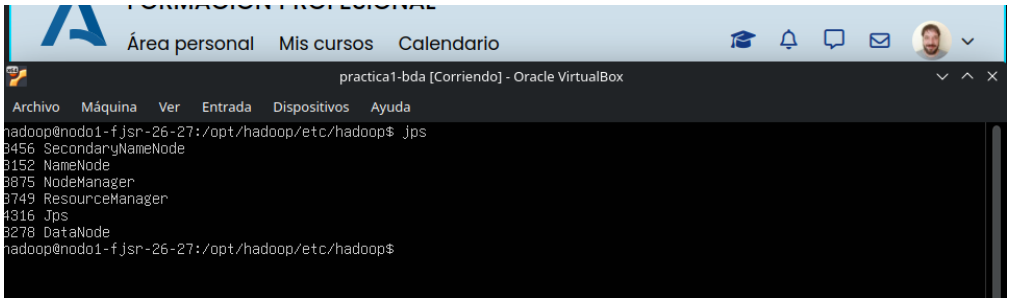


Figura 7.2: Procesos mostrados con jps

8. Interfaces web de Hadoop

Accede a la interfaz del NameNode en **http://IP_DEL_NODO:9870** y a YARN ResourceManager en **http://IP_DEL_NODO:8088**. Explica brevemente qué información ofrece cada interfaz.

8.1. Solución

En este ejercicio nos hemos conectado a las dos interfaces web que proporciona, tanto **NameNode** como **YARN**. Ya que en la máquina virtual tenemos instalada **Ubuntu Server**, que no viene con entorno gráfico por defecto, se ha realizar la conexión desde el sistema host, configurando los puertos desde la máquina virtual. Las 2 interfaces ofrecen información diferente y permiten realizar cosa diferentes, como vemos a continuación:

- **Interfaz de NameNode**: esta interfaz nos permite acceder a información y gestionar el sistema de archivos HDFS, viendo el estado general, el número de nodos, se ha habido errores, consultar los logs, ...etc. En la siguiente captura podemos ver la interfaz.

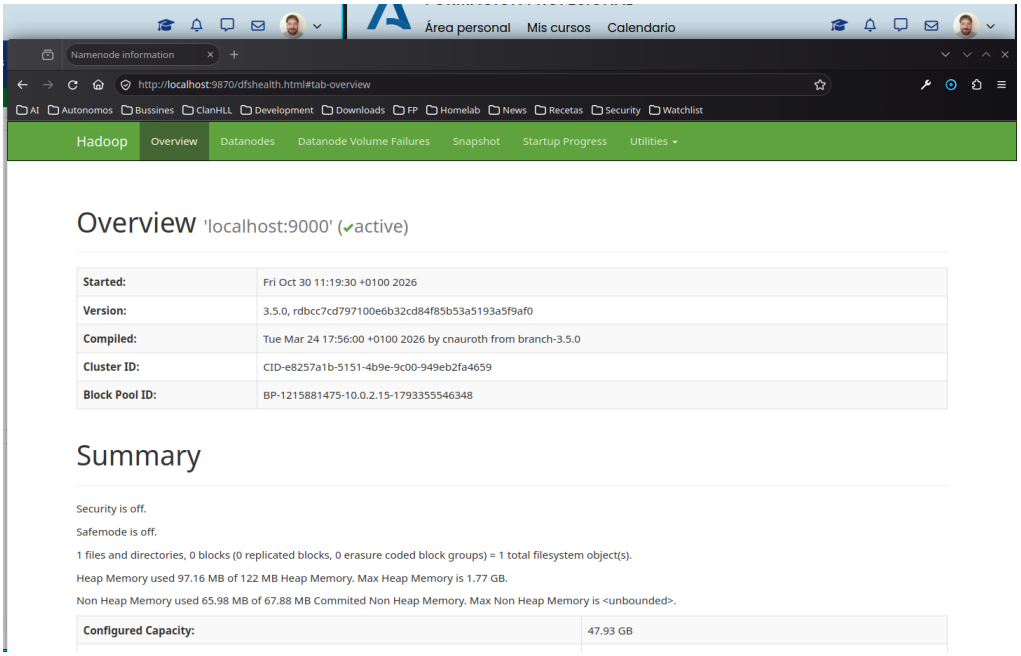


Figura 8.1: Interfaz web de NameNode

- **Interfaz de YARN**: por otro lado, la interfaz de YARN nos permite gestionar los recursos de la red de nodos y de las aplicaciones que se ejecutan en ellas, visualizando estadísticas, aplicaciones, planificación, ...etc. La interfaz es la siguiente:

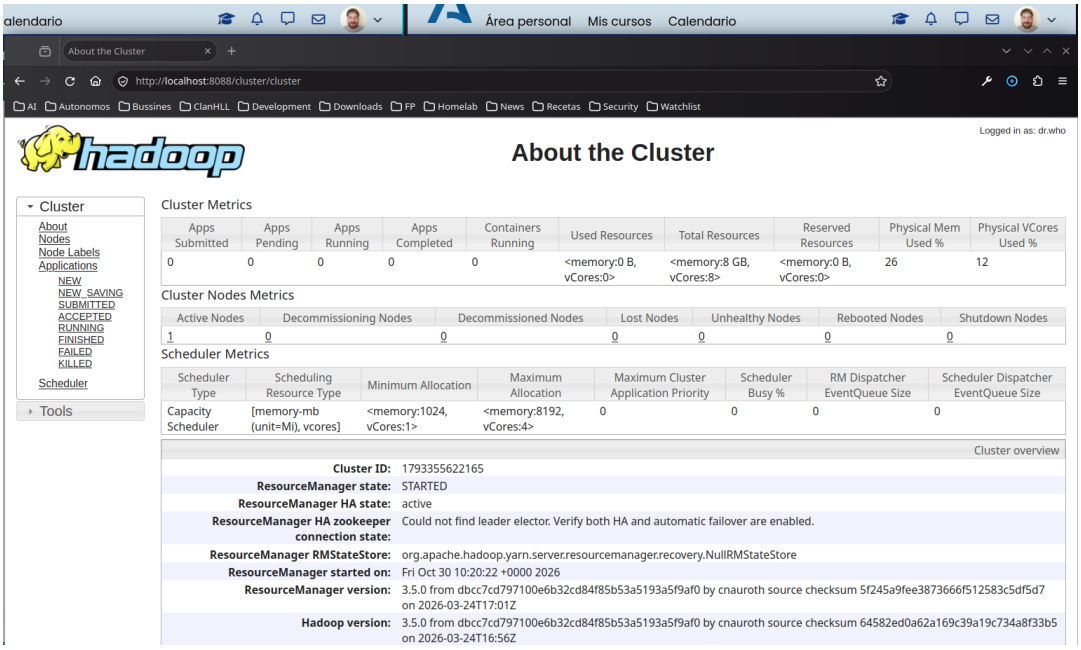


Figura 8.2: Interfaz web de YARN

9. Sistemas de almacenamiento Big Data

Redacta una explicación de 300-500 palabras sobre HDFS, NameNode, DataNode, bloques, replicación, diferencias con un sistema de ficheros tradicional, escalabilidad y ventajas de aumentar el número de nodos. Relaciónalo con la instalación realizada.

9.1. Solución

HDFS (Hadoop Distributed File System) es el sistema de archivos que usa Apache Hadoop, un sistema de archivos distribuido diseñado para almacenar grandes cantidades de información repartidas entre diferentes equipos o nodos. Su objetivo principal es ofrecer un sistema escalable y eficiente, a la par que tolerante a fallos, capaz de trabajar con grandes cantidades de datos de forma eficiente.

Este sistema de archivos se basa principalmente en dos componentes: **NameNode** y **DataNode**. El **NameNode** es el encargado de gestionar la **metainformación** de los archivos, es decir, información sobre donde se ubican los archivos, que permisos tienen, los directorios, etc. Mientas que por otro lado, **DataNode** se encarga de almacenar la información que contienen los archivos propiamente dicha, así como de gestionar la operaciones de lectura y escritura.

La unidad en la que se almacenan los archivos en los DataNodes son los **bloques**. Un archivo se divide en diferentes bloques que pueden ser almacenados en diferentes nodos, incluso el mismo bloque en varios nodos, utilizando lo se que conoce como **replicación**, una técnica que usa Hadoop para que un mismo bloque se almacene en diferentes nodos, de esta forma, se consigue un sistema, muy tolerante a fallos, ya que si un nodo deja de estar disponible, por ejemplo, se podrá recuperar desde otro nodo esa misma información.

Esta arquitectura favorece la **escalabilidad horizontal** del sistema, ya que simplemente añadiendo más nodos podemos expandirlo, permitiendo que la cantidad de datos que se pueden almacenar en el sistema aumente, en este aspecto, **aumentar el número de nodos** beneficia la distribución de carga en el sistema, así como la tolerancia a fallos, ya que podemos tener los bloques copiados en más nodos, además de facilitar el crecimiento de la estructura en general.

Esto, es **muy diferente** a un **sistemas de archivos tradicional**, ya que estos sistemas almacenan toda la información en un mismo ordenar, mientras que HDFS lo hace un multitud de nodos o equipos.

En **nuestra instalación** no disponemos de varios equipos, por eso hemos realizado una configuración **pseudo-distribuida**, que aunque todo se ejecuta en una misma máquina, se mantienen los procesos separados, simulando su ejecución en diferentes nodos. Además, hemos indicado la opción **dfs.replication = 1**, ya que solo disponemos de una DataNode, y no tiene dinciar que se mantengan copias de los bloques en otros nodos.

10. Creación de la Estructura HDFS

Crea en HDFS **/datos**, **/datos/entrada**, **/datos/csv**, **/datos/json** y **/resultados**. Comprueba la estructura con **hdfs dfs -ls /** y **hdfs dfs -ls /datos**.

10.1. Solución

En este ejercicio se han creado los directorios pedidos empleando el comando **hdfs dfs -mkdir**. Este comando no creo directorios en el sistema de archivos del SO, sino dentro de HDFS. En la siguiente captura se puede ver la estructura de archivos creados.

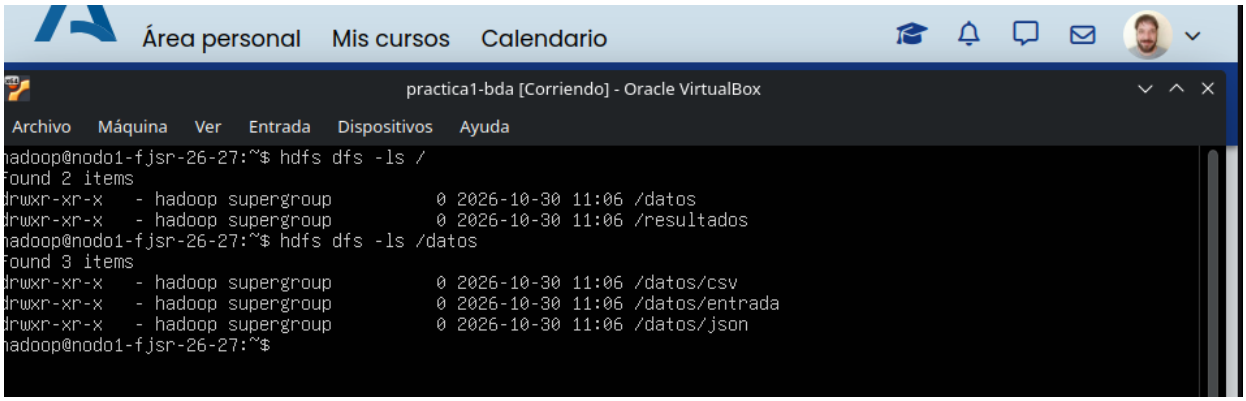


Figura 10.1: Estructura de archivos en HDFS

11. Preparación del conjunto de datos

Utiliza el fichero **ventas.csv** suministrado, que contiene 100 registros más una cabecera. Identifica sus campos y comprueba el número de líneas mediante **wc -l ventas.csv**.

11.1. Solución

En primer lugar se ha comprobado que el fichero tiene las líneas que debe tener, como podemos ver en la siguiente captura.

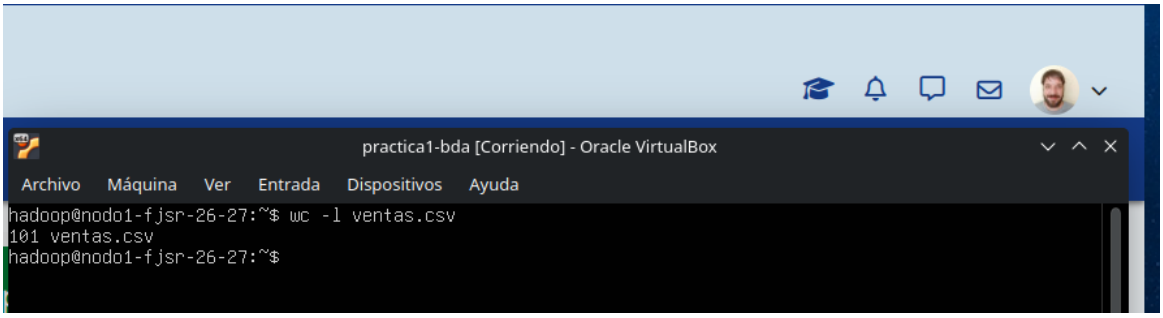


Figura 11.1: Conteo de líneas del archivo ventas.csv

Además, ha abierto el fichero **ventas.csv**, viendo que los campos que tiene son: **id**, **fecha**, **provincia**, **categoría**, **producto**, **cantidad** y **precio**, como podemos ver.

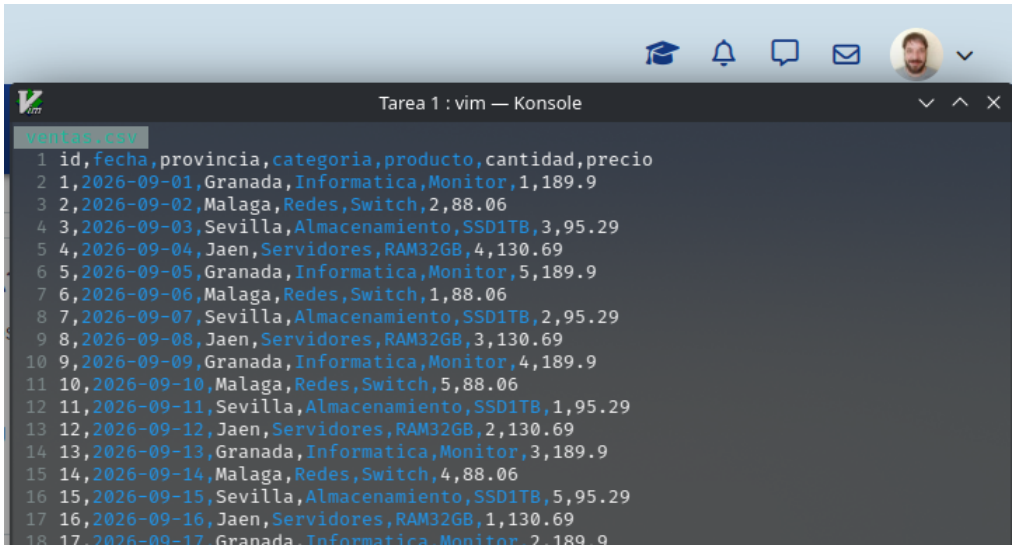


Figura 11.2: Archivo ventas.csv

12. Ingestión de los Datos

Realiza la ingestión del fichero local en HDFS usando y explicando comandos como **hdfs dfs -put**, **hdfs dfs -copyFromLocal**, **hdfs dfs -ls** y **hdfs dfs -cat**. Explica la diferencia entre **put** y **copyFromLocal**.

12.1. Solución

En este ejercicio hemos realizado la **ingestión de los datos**. Esto consiste en pasar los datos, en nuestro caso por lotes, al sistema de archivos HDFS. En primer lugar, hemos copiado los archivos a la máquina virtual, empleando ssh. Se han copiado tanto los archivos ventas.csv como ventas.json, así ya lo tenemos para el próximo ejercicio. En la siguiente captura, se muestran los 2 archivos en el directorio home del usuario hadoop.

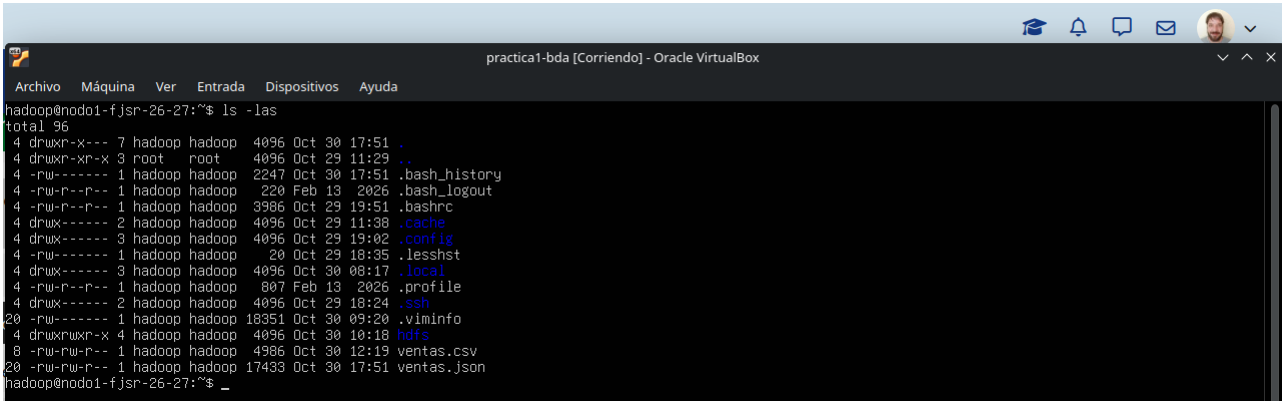


Figura 12.1: Archivos locales de datos

Una vez copiados los archivos, hemos usado el comando **hdfs dfs -put** para copiarlos al sistema

HDFS. Tanto este comando como **hdfs dfs -copyFromLocal** son muy parecidos, la diferencia es que mientras que **copyFromLocal** esta restringido a archivos locales, mientras que **put** puede realizar la copia desde otras fuentes, pero en la práctica, son iguales.

Para comprobar si el archivo estaba en el sistema de ficheros HDFS, hemos usado el comando **hdfs dfs -ls**. Es comando es **similar a ls de Unix/Linux**, es un comando que se limita a listar los elementos que hay en un directorio. De igual manera, para ver el contenido del fichero se ha usado **hdfs dfs -cat**, este comando, al igual que el **comanto cat de Linux**, concatena un fichero o un stream a la salida estándar, lo que en la práctica muestra el contenido de un fichero de texto o los bytes de un fichero binario. En la siguiente captura podemos ver la salida de estos dos comando.

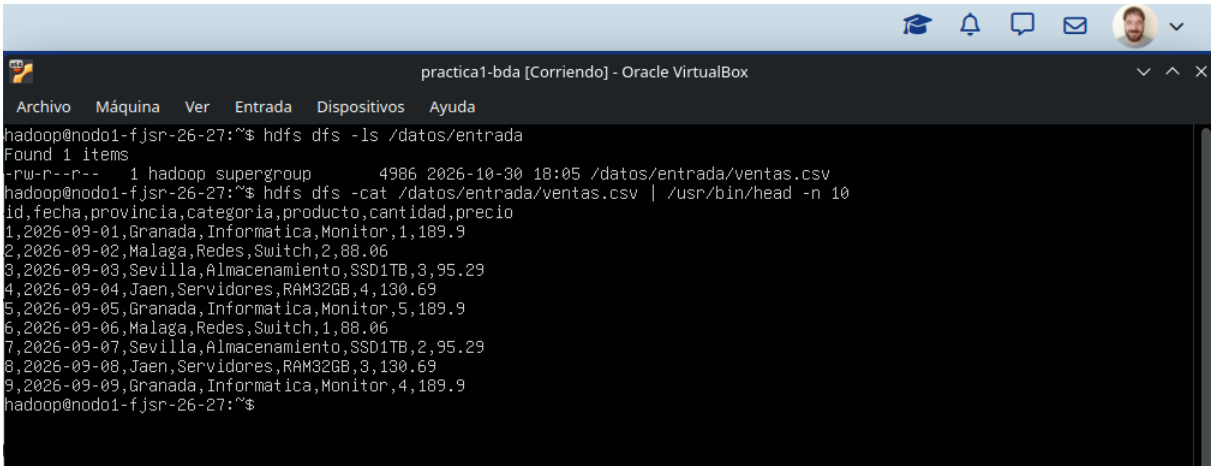


Figura 12.2: Uso de cat y ls en HDFS

La **salida de cat se a limitado a 10 líneas**, para que se puedan ver los dos comandos en la captura pero demostrando su funcionamiento.

13. Análisis de formatos de Almacenamiento

Trabaja con **ventas.csv** y su equivalente **ventas.json**. Compara estructura, tamaño, legibilidad, tipado, datos anidados, facilidad de procesamiento y casos de uso. Comprueba tamaños con **ls -lh** y **hdfs dfs -du -h /datos**. Justifica qué formato elegirías para este caso.

13.1. Solución

En ese ejercicio se va a realizar una comparación entre los dos formatos en los que nos encontramos los datos en nuestra tarea, a saber, **CSV** y **JSON**.

En primer lugar habría que hablar de **CSV** (Comma Separated Values), un formato antiguo y muy simple que consta simplemente de un **conjunto de valores**, agrupados por filas (o no), **separados por una coma**. Se puede incluir una cabecera si se quiere especificar que representa cada valor. Tiene la ventaja de que es **muy simple** y ocupa **poco espacio**. En la siguiente captura podemos ver el contenido de nuestro archivo como ejemplo.

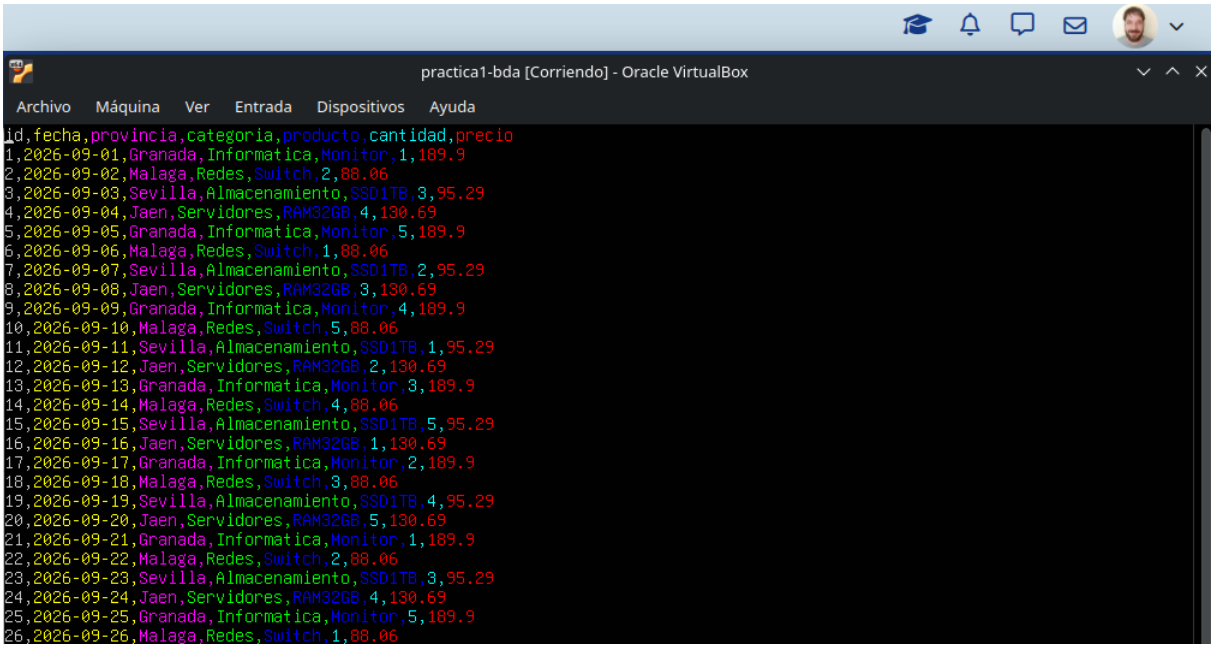


Figura 13.1: Contenido del fichero en formato CSV

Por otro lado tenemos a **JSON** (JavaScript Object Notation), un formato que surge del lenguaje de programación Javascript, ampliamente usado, y que se compone de pares **clave/valor**, de una forma estructura. Este formato, se estructura en **objetos**, que se pueden anidar los objetos, además de incluir arrays de valores. Aunque es un formato que ocupa más espacio, es más **fácil de leer para los humanos** y de procesar, ya que cada valor tiene asociado una clave. En la siguiente captura podemos ver un ejemplo con nuestro archivo ventos.json.

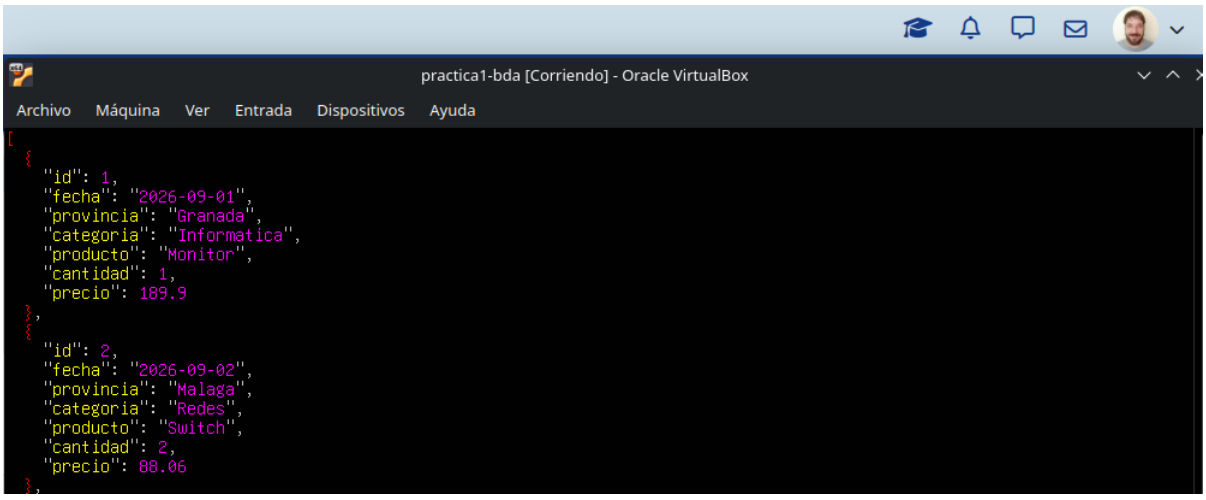


Figura 13.2: Contenido del fichero en formato JSON

Como podemos ver, cada uno de los **formatos** tiene su **ventajas**, cuando la cantidad de datos no es muy grande y no pude haber variaciones, o cuando el espacio de almacenamiento es crítico, **CSV** puede ser una buena opción, aunque como podemos comprobar, esta mucho más limitado, el espacio que ocupa

la misma información es 1/3 de lo que ocupa esa misma información en formato json, como podemos ver en la siguiente captura, donde se muestra el tamaño de los directorios donde se encuentra los respectivos archivos.

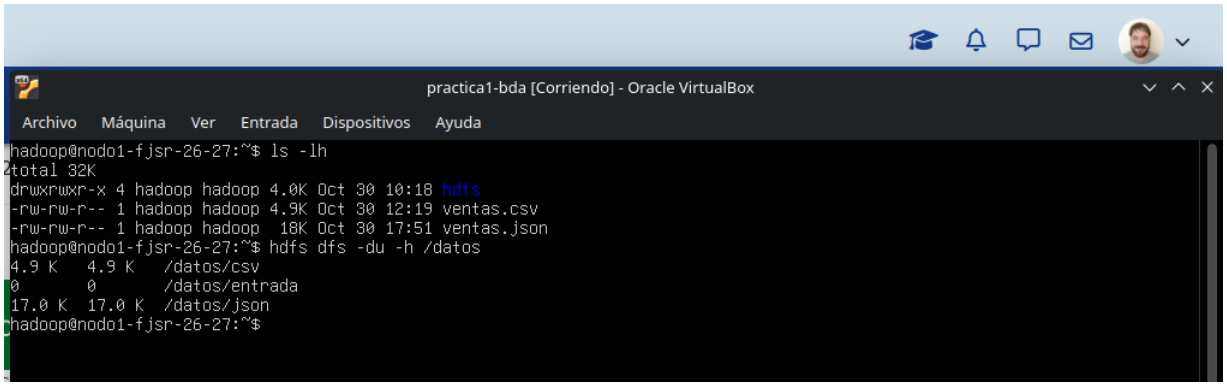


Figura 13.3: Tamaño de los archivos CSV y JSON en local y dentro de HDFS

Personalmente, y **dado el volumen** de datos y que estos son **relativamente simples**, escogería **CSV** como formato, a pesar de que soy un firme defensor de JSON, pero creo que sería más adecuado para nuestro propósito.

14. Comprobación de almacenamiento físico en HDFS

Desde la interfaz web del NameNode localiza **ventas.csv** e identifica tamaño, número de bloques, identificador de bloque, DataNode y factor de replicación. Relaciona los valores con **hdfs-site.xml**.

14.1. Solución

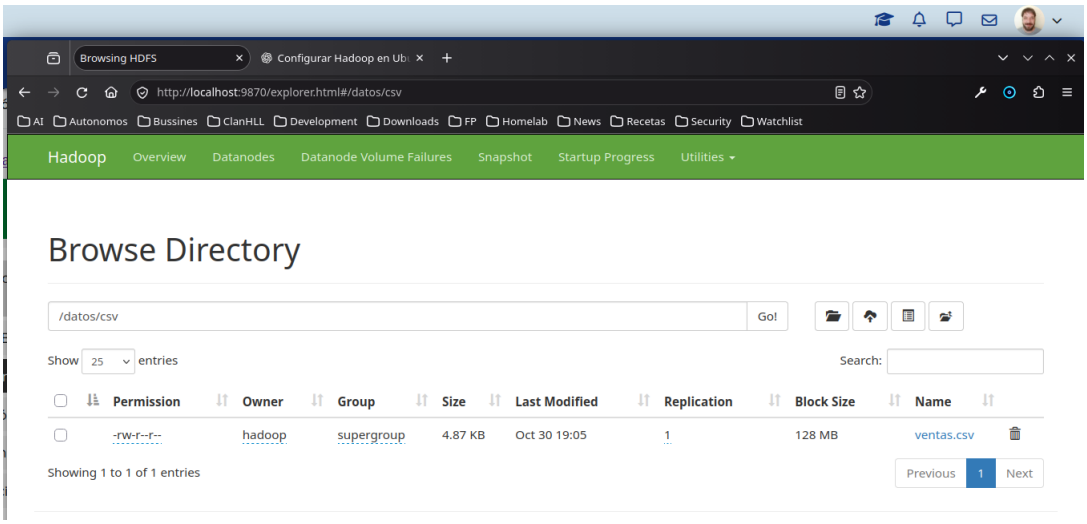


Figura 14.1: Información del fichero ventas.csv en la interfaz web de NameNode

En este ejercicios vamos a comprobar el espacio físico que ocupa nuestro fichero ventas.csv en el sistema de archivos HDFS. Para comprobarlo, podemos consultar la **interfaz web de NameNode**, y utilizando los utilidad **Browse the file system** buscar el archivo y ver su información.

Como podemos ver en la captura que la interfaz nos da información de los **permisos**, **propietario**, **ultima modificación**, **replicación**, **tamaño de bloque** y el **nombre**. El **tamaño** del archivo es de **4.87KB**, es no ocupa ni un 1 % de un bloque, que como podemos ver tienen un tamaño de 128MB. Pulsando en el fichero, podemos ver el resto de datos que se nos piden, como podemos ver en la siguiente captura.

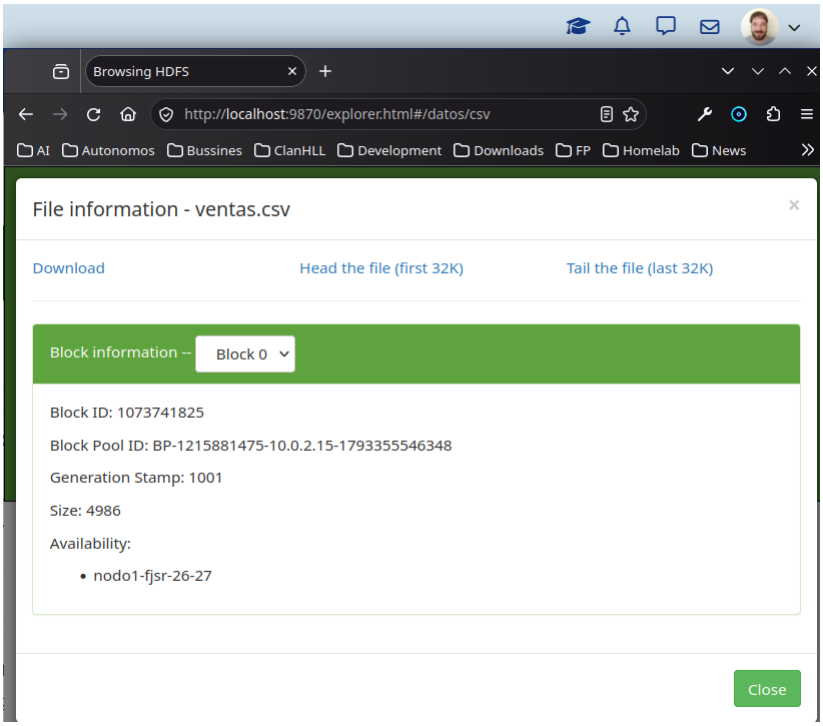


Figura 14.2: Detalle el fichero ventas.csv en la interfaz web de NameNode

En esta ventana, podemos ver el nombre del nodo, **nodo1-fjsr-26-27**, así como su ID, su tamaño, ..etc. Como podemos ver, el **factor de replicación**, que aparece en la primera captura, esta **establecido en 1**, que exactamente el valor que hemos configurado en el fichero **hdfs-site.xml**.

15. Procesamiento de los datos

Obtén: (1) total de registros, (2) operaciones por provincia, (3) importe total por provincia usando **cantidad * precio**, (4) operaciones por categoría y (5) provincia con mayor volumen económico.

15.1. Solución

Para este ejercicio, debemos obtener diferentes datos de nuestro archivo. Dado que aún no he visto nada de Python, y ciertamente yo no he programado nunca, se va a usar, como se ha sugerido en el foro **AWK**, un comando con el si estoy más familiarizado, ya que soy usuarios de Linux desde hace muchos años, y para mostrar los datos y realizar todo de forma más .elegante", se va integrar en un script de Bash.

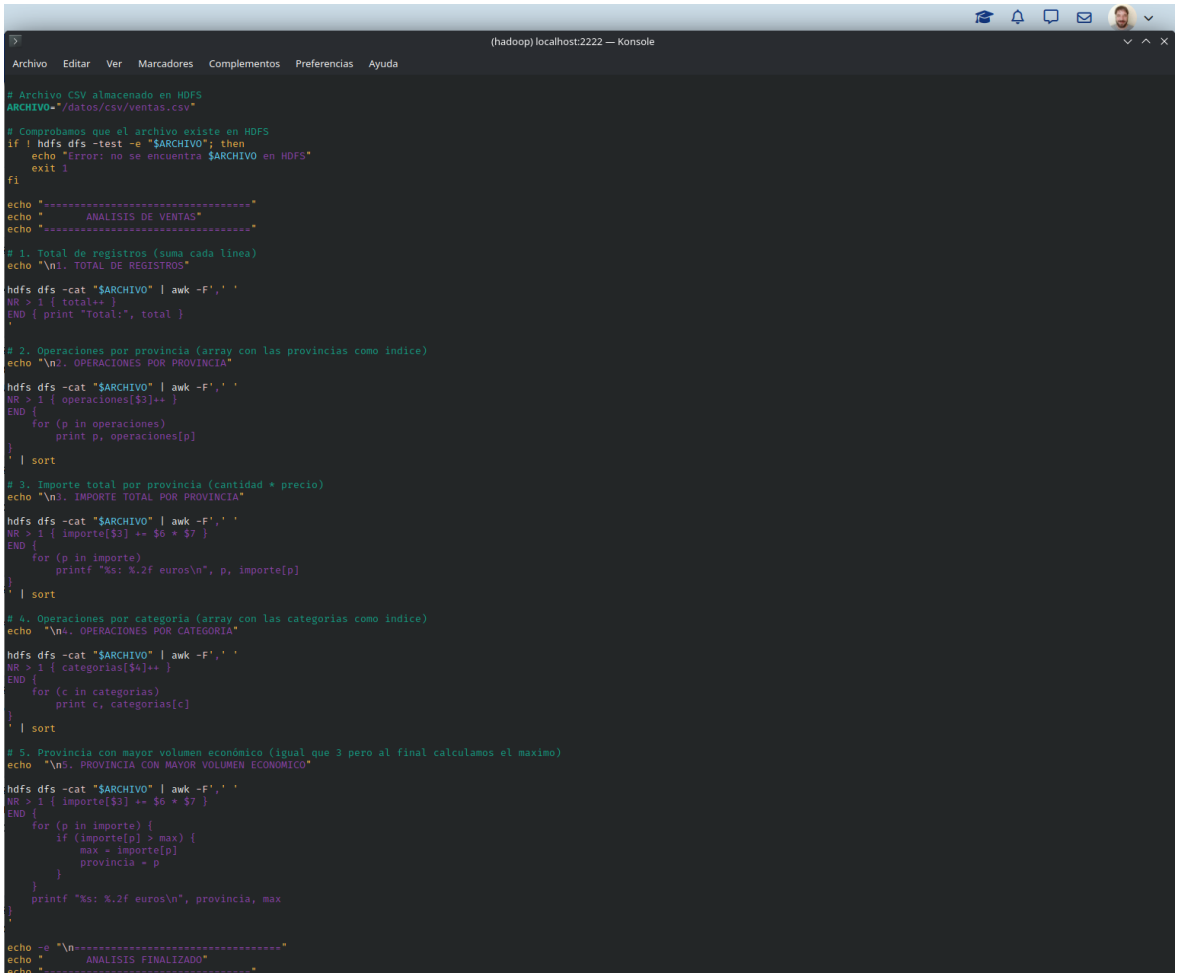


Figura 15.1: Script de bash para procesar los datos

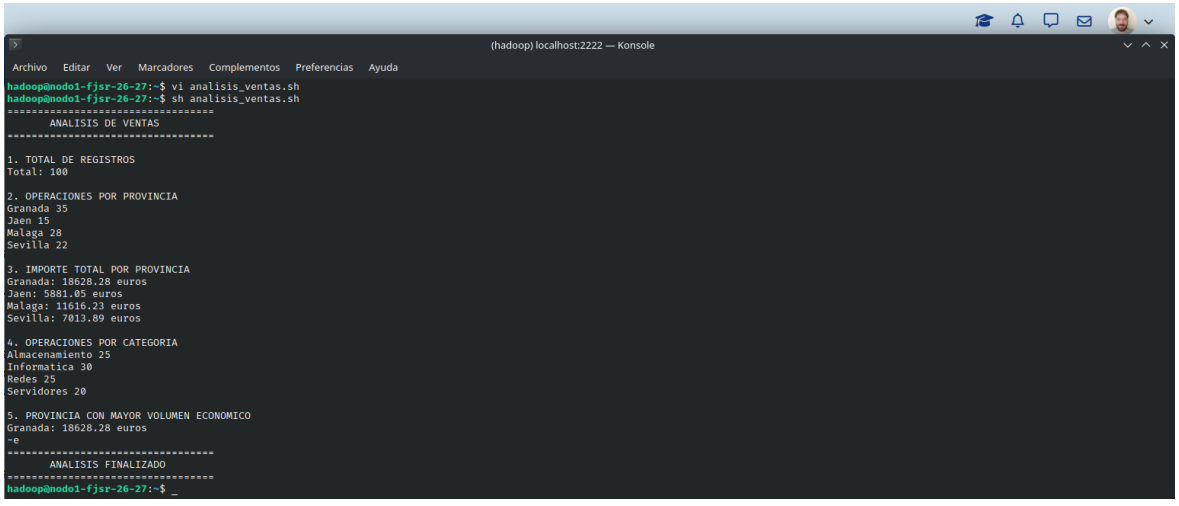


Figura 15.2: Resultado del procesamiento de los datos.

Como podemos ver el script es bastante simple, simplemente **ejecutamos una acción** cada vez que estamos en una **línea por debajo de la cabecera** (NR > 1), que suele consistir en crear un array cuyos

índices sean los elementos que queremos contar y los valores el valor en sí mismo. Después de acabar (bloque END), procesamos el array el mostramos y los resultados.

Referencias

[1] RFC 1123.
https://www.rfc-editor.org/info/rfc1123/.

[2] IBM HDFS.
https://www.ibm.com/think/topics/hdfs.